Data Structure and algorithms (LAB)

Lab Task

Name: UzairAlam

PROBLEM 1

```
#include <iostream>
using namespace std;

struct Node
{
   string id;
   Node *next;
   Node *pre;

   Node() : id(""), next(nullptr), pre(nullptr) {}
   Node(const string &id) : id(id), next(nullptr), pre(nullptr) {}
};

class doubleLinkList
{
   Node *head, *tail;

public:
   doubleLinkList() : head(nullptr), tail(nullptr) {}

   void insertAtBegin(const string &id)
   {
      Node *newNode = new Node(id);
      if (head == nullptr)
```

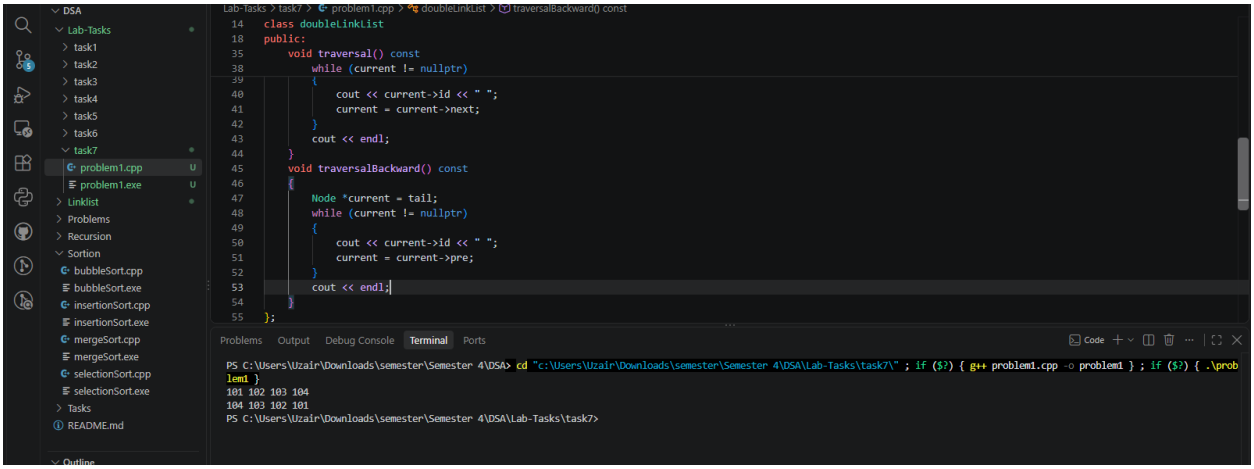
```
    {
        head = newNode;
        tail = newNode;
        return;
    }
    newNode->next = head;
    head->pre = newNode;
    head = newNode;
}

void traversal() const
{
    Node *current = head;
    while (current != nullptr)
    {
        cout << current->id << " ";
        current = current->next;
    }
    cout << endl;
}
void traversalBackward() const
{
    Node *current = tail;
    while (current != nullptr)
    {
        cout << current->id << " ";
        current = current->pre;
    }
    cout << endl;
```

```
    }
};

int main()
{
    doubleLinkList list;
    list.insertAtBegin("104");
    list.insertAtBegin("103");
    list.insertAtBegin("102");
    list.insertAtBegin("101");

    list.traversal();
    list.traversalBackward();
}
```



PROBLEM 2

```
#include <iostream>

using namespace std;

struct Node
{
```

```
    string id;
    Node *next;
    Node *pre;

    Node() : id(""), next(nullptr), pre(nullptr) {}
    Node(const string &id) : id(id), next(nullptr), pre(nullptr) {}
};

class doubleLinkList
{
    Node *head, *tail;

public:
    doubleLinkList() : head(nullptr), tail(nullptr) {}

    void insertAtBegin(const string &id)
    {
        Node *newNode = new Node(id);
        if (head == nullptr)
        {
            head = newNode;
            tail = newNode;
            return;
        }
        newNode->next = head;
        head->pre = newNode;
        head = newNode;
    }
```

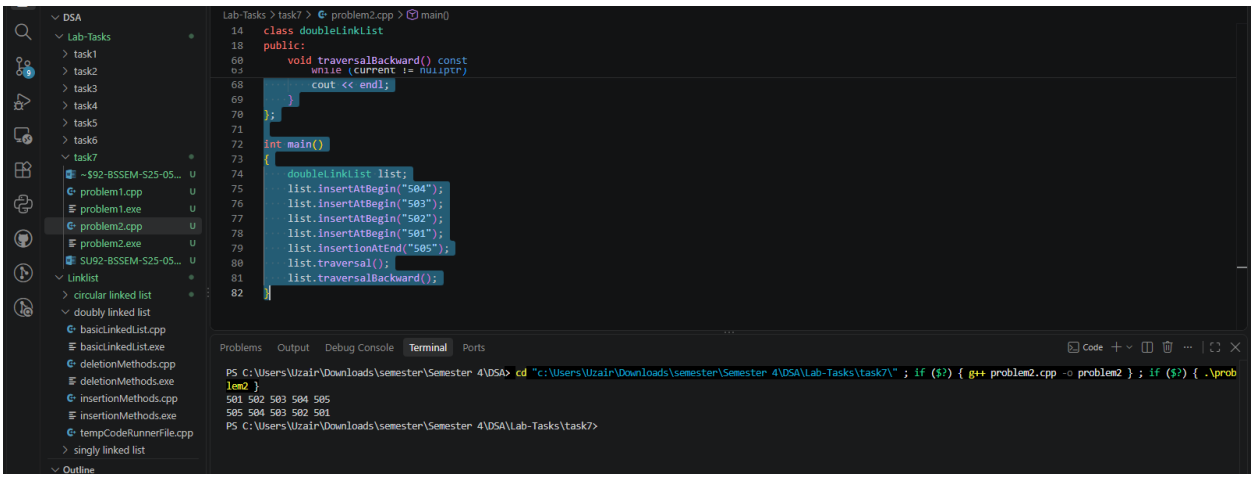
```
void insertionAtEnd(const string &id)
{
   Node *newNode = new Node(id);
   if (head == nullptr)
   {
      head = newNode;
      tail = newNode;
      return;
   }

   tail->next = newNode;
   newNode->pre = tail;
   tail = newNode;
}

void traversal() const
{
   Node *current = head;
   while (current != nullptr)
   {
      cout << current->id << " ";
      current = current->next;
   }
   cout << endl;
}
void traversalBackward() const
{
   Node *current = tail;
   while (current != nullptr)
```

```
        {
            cout << current->id << " ";
            current = current->pre;
        }
        cout << endl;
    }
};

int main()
{
    doubleLinkList list;
    list.insertAtBegin("504");
    list.insertAtBegin("503");
    list.insertAtBegin("502");
    list.insertAtBegin("501");
    list.insertionAtEnd("505");
    list.traversal();
    list.traversalBackward();
}
```



PROBLEM 3

```
#include <iostream>
```

```
using namespace std;

struct Node
{
   string id;
   Node *next;
   Node *pre;

   Node() : id(""), next(nullptr), pre(nullptr) {}
   Node(const string &id) : id(id), next(nullptr), pre(nullptr) {}
};

class doubleLinkList
{
   Node *head, *tail;

public:
   doubleLinkList() : head(nullptr), tail(nullptr) {}

   void insertAtBegin(const string &id)
   {
      Node *newNode = new Node(id);
      if (head == nullptr)
      {
         head = newNode;
         tail = newNode;
         return;
      }
      newNode->next = head;
```

```
    head->pre = newNode;
    head = newNode;
}

void insertAfterTarget(const string &target, const string &id)
{
    Node *current = head;

    while (current != nullptr && current->id != target)
        current = current->next;

    if (current == nullptr)
    {
        cout << "Target Doesn't Exist\n";
        return;
    }

    Node *newNode = new Node(id);

    newNode->next = current->next;
    newNode->pre = current;

    if (current->next != nullptr)
        current->next->pre = newNode;

    current->next = newNode;

    if (current == tail)
        tail = newNode;
```

```
    }

    void traversal() const
    {
        Node *current = head;
        while (current != nullptr)
        {
            cout << current->id << " ";
            current = current->next;
        }
        cout << endl;
    }
    void traversalBackward() const
    {
        Node *current = tail;
        while (current != nullptr)
        {
            cout << current->id << " ";
            current = current->pre;
        }
        cout << endl;
    }
};

int main()
{
    doubleLinkList list;
    list.insertAtBegin("10");
    list.insertAtBegin("20");
```

```
    list.insertAtBegin("40");
    list.insertAtBegin("50");
    list.insertAfterTarget("40", "30");
    list.traversal();
    list.traversalBackward();
}
```

